

PROYECTO FINAL · 5TO PERITO · PROGRAMACION

FlotaSegura GT

Sistema de gestion y seguridad para flotas de transporte pesado en Guatemala

Node.js · TypeScript · MySQL

El problema: fatiga y falta de control en carretera

60%

de accidentes de carga pesada se asocian a fatiga del conductor

0

control automatizado de horas de manejo en flotas pequeñas

24/7

operación continua que exige monitoreo en tiempo real

Las empresas transportistas guatemaltecas necesitan una herramienta simple para vigilar el estado de sus conductores, vehículos y rutas, y reaccionar antes de que un problema se convierta en un incidente.

OBJETIVO DEL PROYECTO

Construir un sistema en TypeScript que administre toda la operacion de una flota de transporte pesado y prevenga accidentes por fatiga, con posibilidad de escalar a una base de datos MySQL real.

- 1 Centralizar empresas, conductores, vehiculos y cargas
- 2 Programar y monitorear viajes en tiempo real
- 3 Detectar fatiga y generar alertas automaticas
- 4 Registrar incidentes y escalar el estado del viaje

Organizacion del codigo por capas



10 entidades del sistema

01

Empresa

Transportistas registradas

02

Conductor

Pilotos y sus licencias

03

Vehiculo

Unidades de la flota

04

Carga

Mercaderia a transportar

05

Ruta

Trayectos origen-destino

06

Punto parada

Paradas dentro de una ruta

07

Viaje

Conductor + vehiculo + ruta +
carga

08

Monitoreo

Telemetria en tiempo real

09

Alerta fatiga

Avisos automaticos de riesgo

10

Incidente

Accidentes y fallas en ruta

Que puede hacer cada modulo

1

Empresas y conductores

Alta, baja, busqueda y actualizacion de datos y estados

2

Vehiculos y cargas

Control de estado, mantenimiento y asignacion de carga

3

Rutas y viajes

Rutas con nivel de riesgo, viajes que unen conductor + vehiculo + carga

4

Monitoreo y alertas

Registro de velocidad y horas continuas; alertas de fatiga automaticas

5

Incidentes

Reporte de eventos; los graves o fatales marcan el viaje como accidente

6

API REST

Modulo de empresas expuesto vía Express en /api/empresas

4 capas de manejo de errores

1

Datos

Si un JSON no existe, se crea vacío; si está dañado, lanza un error claro.

2

Servicios

Valida campos obligatorios y que los ids relacionados existan.

3

Menu

Cada opción vive en un try/catch: el error se muestra y el menú sigue.

4

Global

uncaughtException y unhandledRejection evitan que el programa se caiga.

De archivos JSON a MySQL

El proyecto guarda hoy su informacion en archivos .json (uno por entidad). Para produccion se diseñó database/schema.sql con el modelo relacional completo: 10 tablas con llaves foraneas, tipos ENUM para los estados y datos de ejemplo listos para importar.

```
empresa 1-N conductor / vehiculo / carga
```

```
ruta 1-N punto_parada
```

```
viaje N-1 conductor, vehiculo, ruta, carga
```

```
viaje 1-N monitoreo, alerta_fatiga, incidente
```

schema.sql

```
CREATE DATABASE flotasegura_gt;

CREATE TABLE empresa (
  id VARCHAR(36) PRIMARY KEY,
  nombre VARCHAR(150),
  estado ENUM('activa','inactiva')
);

CREATE TABLE viaje (
  conductor_id VARCHAR(36),
  vehiculo_id VARCHAR(36),
  FOREIGN KEY (conductor_id)
    REFERENCES conductor(id)
);
```


Herramientas utilizadas



TypeScript

Tipado estricto para modelos y servicios



Node.js

Entorno de ejecución del programa



pnpm

Gestor de paquetes del proyecto



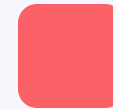
Express

Servidor para el módulo API REST



JSON

Persistencia simple durante el desarrollo



MySQL

Base de datos relacional para producción

CONCLUSIONES

FlotaSegura GT ya controla toda la operacion de una flota, lista para crecer hacia MySQL

SIGUIENTES PASOS

- Agregar los endpoints API para las demas entidades
- Conectar el proyecto a MySQL usando database/schema.sql
- Usar notasClaude para recomendaciones automaticas de riesgo

Gracias — Fundacion Kinal · FlotaSegura GT